

HARARE INSTITUTE OF TECHNOLOGY

REG NO	**H250397M**
COURSE NO	**OOP**
DEGREE PROGRAM	**COMPUTER SCIENCE**
NAME	**LIONEL MUNYAWU**
LAB	

LAB REPORT

I did manage to do all the labs in full and executed them to run to the last output code

Lab 1: Classes and Objects

Objectives

- Define a class with fields (state) and methods (behaviour).
- Create objects with new and access members with the dot operator.
- Understand that object variables hold references, not the objects themselves.
- Recognise null and avoid NullPointerException.

Work done. Wrote a Student class with name, rollNo and gpa plus display() and isHonours(); ReferenceDemo to show aliasing and == on references; and Course, which holds a Student object in a field.

Results. Two students printed correctly, with Tariro (GPA 3.8) honours and Farai (3.1) not. Changing a field through alias b also changed a, a == b was true and a == c was false. A null reference was detected before use.

Conclusion. A class is a blueprint and an object is one instance of it on the heap. Reference variables store addresses, so assignment copies the reference and not the object. Checking for null before use prevents runtime failures.

Lab 2: Constructors, the this Keyword and static Members

Objectives

- Initialise objects safely using constructors.
- Overload constructors and chain them with this(...).
- Use this to distinguish fields from parameters.
- Explain the difference between instance members and static members.
- Use static and instance initialiser blocks.

Work done. Built a Book class with three overloaded constructors chained through this(...), a shared static counter and both initialiser blocks. Also wrote InitOrder to trace initialisation order and a Point class with a copy constructor.

Results. Three books received ids 1 to 3 and Book.getCount() returned 3. The static block ran once, before any object was created. In InitOrder, static parts ran once and instance parts ran again for every new object. The copy constructor produced an equal but separate Point.

Conclusion. Constructors guarantee that an object starts in a valid state, and chaining removes duplicated setup code. Static members belong to the class and are shared, while instance members belong to each object. Initialisation always follows the order: static, then instance blocks, then the constructor body.

Lab 3: Encapsulation and Access Modifiers

Objectives

- Hide internal data with private fields and expose it through controlled methods.
- Apply the four access levels correctly.
- Enforce validation rules (class invariants) in setters and constructors.
- Design classes that cannot be put into an invalid state.

Work done. Implemented a BankAccount with private fields, validated constructor and setter, and no setBalance(), so money moves only through deposit() and withdraw(). Then wrote Team to compare a getter that leaks its internal list with one that returns an unmodifiable view.

Results. A deposit of 250 gave a balance of 750.00, a withdrawal of 100 succeeded, and a withdrawal of 5000 was refused with the balance unchanged. A negative deposit was rejected with an exception. The unsafe getter let a caller empty the team, while the safe getter threw UnsupportedOperationException.

Conclusion. Private fields with validated access methods keep an object consistent and let internals change without breaking callers. Returning references to mutable internals breaks encapsulation, so getters should return copies or read-only views.

Lab 4: Inheritance

Objectives

- Create subclasses with extends and reuse superclass code.
- Call superclass constructors and methods with super.
- Use protected, final and @Override correctly.
- Explain constructor call order and the role of java.lang.Object.
- Decide between inheritance ("is-a") and composition ("has-a").

Work done. Built an Employee hierarchy with Manager and Developer subclasses that override calculatePay(). Traced constructor order through a three-level chain A, B, C, and modelled a Car that contains an Engine as a composition example.

Results. Superclass constructor messages printed before subclass messages. The inherited describe() gave 2000.00 for the employee, 4300.00 for the manager and 3250.00 for the developer, each using its own pay logic. The chain printed A, B, C in order.

Conclusion. Inheritance reuses and specialises code for true "is-a" relationships, with the parent constructor always running first. @Override catches signature mistakes at compile time, and composition is the better choice when the relationship is "has-a".

Lab 5: Polymorphism

Objectives

- Distinguish compile-time polymorphism (overloading) from run-time polymorphism (overriding).
- Use a superclass reference to hold subclass objects and explain dynamic method dispatch.
- Downcast safely with instanceof and pattern matching.
- Write code that works with new subclasses without modification.

Work done. Overloaded add in a Calculator for ints, doubles and strings. Created Shape with Circle, Rectangle and Triangle subclasses stored in one array, and tested casting, field hiding and static method hiding with Animal and Dog.

Results. Each overloaded call chose the right version from its argument types. The shape loop printed areas of 78.54, 24.00 and 12.00, totalling 114.54, without knowing the concrete types. A Dog held in an Animal variable barked, but its field and static method followed the declared type.

Conclusion. Overloading is resolved by the compiler and overriding by the JVM from the actual object. Only instance methods are polymorphic, so fields and static methods follow the declared type. Polymorphism lets new subclasses be added without changing the code that uses them.

Lab 6: Abstraction, Abstract Classes and Interfaces

Objectives

- Define abstract classes and abstract methods, and explain why they cannot be instantiated.
- Define and implement interfaces, including default and static methods.
- Choose correctly between an abstract class and an interface.
- Program to an interface, not to an implementation.
- Use standard interfaces such as Comparable.

Work done. Created an abstract Vehicle with Truck and Motorbike subclasses. Wrote Payable and Loggable interfaces with default and static methods, implemented by CardPayment and CashPayment. Made a Player class implement Comparable to sort by score.

Results. A 100 km trip cost 80.00 by truck and 15.00 by motorbike through one shared method. CardPayment used the default receipt, CashPayment overrode it, and a 50,000 payment was blocked by the limit check. Players sorted as Blessing (95), Chenai (82), Anesu (70).

Conclusion. Abstract classes suit closely related types that share code and state, while interfaces define capabilities that unrelated classes can share and allow multiple implementation. Programming to the interface keeps code flexible, and library interfaces such as Comparable plug custom classes into standard tools.

Lab 7: Exception Handling

Objectives

- Explain the exception hierarchy and the difference between checked and unchecked exceptions.
- Use try, catch, finally, throw and throws correctly.
- Create custom exceptions that carry useful information.
- Manage resources safely with try-with-resources.
- Write methods that fail predictably instead of leaving objects in a bad state.

Work done. Handled division by zero, bad array indexes, null references and invalid numbers with ordered catch blocks and multi-catch. Created a checked InsufficientFundsException used by a Wallet class, and an AutoCloseable Connection class to demonstrate try-with-resources and exception chaining.

Results. finally ran whether or not an error occurred. The wallet refused a second spend of 60 with a message and a shortfall of 20.0. Resources closed in reverse order (DB2 then DB1) even though a query failed, and a wrapped exception kept its original cause.

Conclusion. Exceptions separate normal logic from error handling and let failures be handled where they can be fixed. Specific catch blocks, custom exceptions with context, and try-with-resources make programs more robust and prevent resource leaks.

Lab 8: Generics and the Collections Framework

Objectives

- Write generic classes and methods, including bounded type parameters.
- Explain type erasure and why primitives need wrapper classes.
- Choose the right collection: List, Set, Map, Queue.

- Sort and search with Comparable, Comparator and Collections.
- Program to the interface, for example List<String> x = new ArrayList<>().

Work done. Wrote generic Box<T> and Pair<K,V> classes plus generic methods printAll and a bounded maxOf. Used ArrayList, TreeSet, HashMap, TreeMap and ArrayDeque for lists, sets, maps, a word-frequency count, a stack and a queue. Sorted Student objects by name with Comparable and by GPA with Comparator.

Results. Generic types caught errors at compile time and removed casts. The list kept duplicates while the set removed them, a repeated map key replaced its value, and the word count gave {be=2, not=1, or=1, to=2}. The stack popped 3 (last in, first out) and the queue returned "first" (first in, first out).

Conclusion. Generics provide type safety and reusable code, and each collection fits a purpose: lists for order, sets for uniqueness, maps for lookup by key and queues for processing order. Comparators allow many sort orders, while Comparable gives one natural order.

Lab 9: Nested Types, Enums, Records and the Object Contract

Objectives

- Distinguish static nested, inner, local and anonymous classes.
- Model fixed sets of constants with enum, including fields, constructors and methods.
- Create immutable data carriers with record.
- Override equals, hashCode and toString correctly and explain why they must be consistent.

Work done. Wrote one program with all four kinds of nested class, including an inner iterator that reads the outer object's private data. Built enums Day, Planet and Operation (with constant-specific methods), a Point class with equals/hashCode/toString, and a Coordinate record with a validating constructor.

Results. The inner class read outer data (10 20 30) and the static nested class reported instances created. Enum constants worked in a switch and Earth gravity printed as 9.80 m/s². Two equal Point objects were != by reference but equals, and a HashSet kept only one. The record generated Coordinate[x=3, y=4] and distance 5.0.

Conclusion. Nested classes keep helper types next to the code that uses them. Enums give type-safe constants, and records remove boilerplate for immutable data. equals and hashCode must be overridden together or objects misbehave in hash-based collections.

Lab 10: File I/O, Lambdas and Streams

Objectives

- Read and write text files with java.nio.file and handle IOException.
- Persist and restore objects using a simple CSV format.
- Write lambda expressions and method references, and identify functional interfaces.
- Process collections declaratively with the Stream API (filter, map, sorted, collect, reduce).

Work done. Wrote, appended to and read back students.txt, and handled a missing file separately. Saved and reloaded Learner records through a StudentRepository using CSV lines. Used Predicate, Function, Consumer and Supplier lambdas, then ran stream pipelines over a list of Employee records.

Results. Four records were written and read back, and the missing file produced a clear message instead of a crash. Saved learners reloaded as identical objects. The streams listed high earners Ada, Ben and Dan, showed a total of 19400 with an average of 3880.0, and grouped head count and average pay by department.

Conclusion. The java.nio.file API with proper exception handling makes data persistent and reliable. Lambdas and streams express filtering, transforming and grouping data in a few readable lines, without changing the source collection.

Lab 11: Design Patterns and the Mini-Project

Objectives

- Recognise four common patterns (Singleton, Factory, Strategy, Observer) and the problem each one solves.
- Apply SOLID-style thinking: single responsibility, open for extension, program to interfaces.
- Combine classes, interfaces, collections, exceptions and file I/O in one working application.

Work done. Implemented a Singleton AppConfig, a NotificationFactory that returns email or SMS senders, a Checkout class with swappable discount strategies, and a GradeBook that notifies subscribed listeners. Then planned the Library Management System mini-project using an abstract LibraryItem, a Borrowable interface, custom exceptions, map-based storage, CSV persistence and a stream-based search.

Results. Both getInstance() calls returned the same object. The factory created the correct notification type from a string. Switching strategy changed 200 to 180 (a 10% discount) at run time, and one grade publication reached both the student portal and the advisor, who was alerted for a mark of 45.

Conclusion. Design patterns are named, reusable solutions: Singleton controls instances, Factory hides concrete classes, Strategy swaps behaviour and Observer keeps objects loosely coupled. The mini-project shows how encapsulation, inheritance, polymorphism, abstraction, exceptions, collections and file handling work together in one application.

Overall Conclusion

The eleven labs moved from single classes to a complete application, and together they met the course outcomes.

- Labs 1 to 3 showed how to model entities as classes and objects and protect their state through encapsulation.
- Labs 4 to 6 applied inheritance, polymorphism and abstraction to build flexible, extensible designs.
- Labs 7 to 10 added robustness and productivity through exceptions, generics, collections, enums, records, file handling, lambdas and streams.
- Lab 11 combined these ideas using design patterns and the library mini-project.

The main lesson is to keep data private, depend on interfaces, prefer composition unless the relationship is truly "is-a" and handle errors deliberately. These habits make Java programs easier to test extend and maintain.

So this is why i did the lab conclusion

NB some of the labs where assisted or were partnered by artificial intelligence

The most beneficial lab i found was the bubble sort which assisted me to write the data algorithm sorting which is implemented in real world day to day application